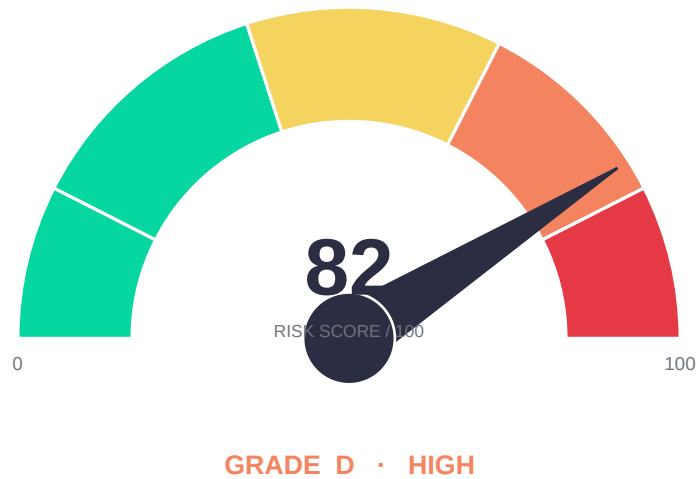


Penetration Testing Report

20/06/2026



Prepared by:

RMW-PenStrike

Email:

contact@rmw-penstrike.com

This document is classified CONFIDENTIAL. It may not be copied without written permission.

Table of Contents

1. Executive Summary

1.1 Assessment Summary

1.2 Strategic Recommendations

2. 1. Technical Summary

2.1 1.1 Scope

2.2 1.2 Post Assessment Clean-up

2.3 1.3 Risk Ratings

2.4 1.4 Findings Overview

3. 2. Technical Details

3.1 2.1 Hardcoded Credentials in Login Page — HIGH

3.2 2.2 Path Traversal to Sensitive File — HIGH

3.3 2.3 Reflected Cross-Site Scripting via eval() Injection — HIGH

3.4 2.4 Privilege Escalation via Client-Side Authorization — HIGH

4. 3. Appendices

4.1 3.1 Penetration Testing Methodology

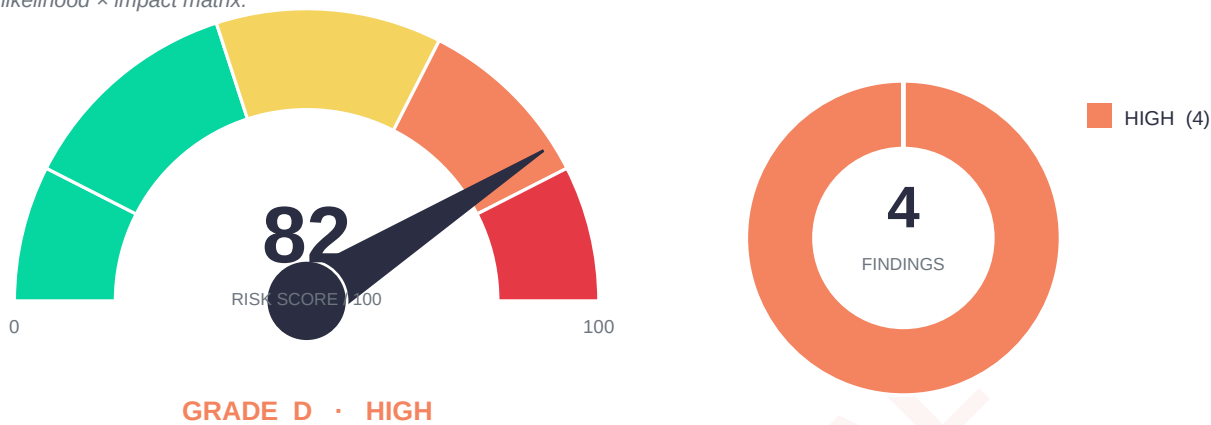
Threat Intelligence & Public Exploits

Remediation Roadmap

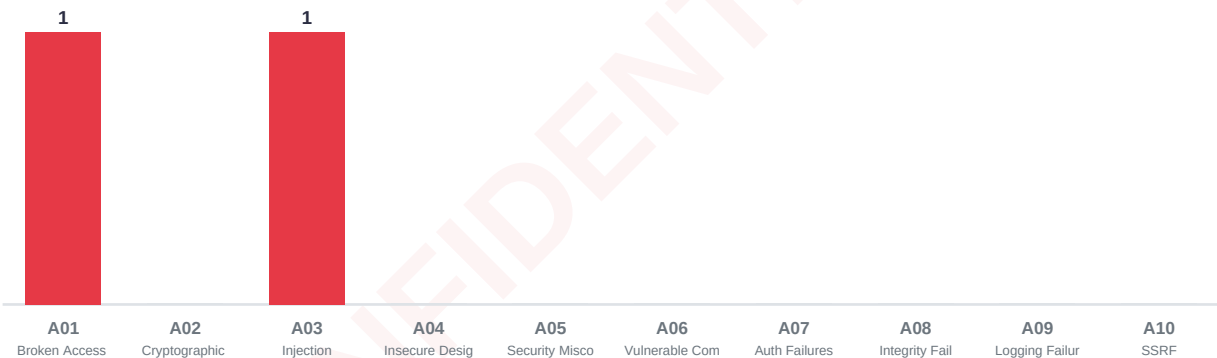
CONFIDENTIAL

Risk Posture

At-a-glance security posture derived from all findings: overall risk score, severity distribution, OWASP Top 10 coverage, and a likelihood × impact matrix.



Findings by OWASP Top 10 Category



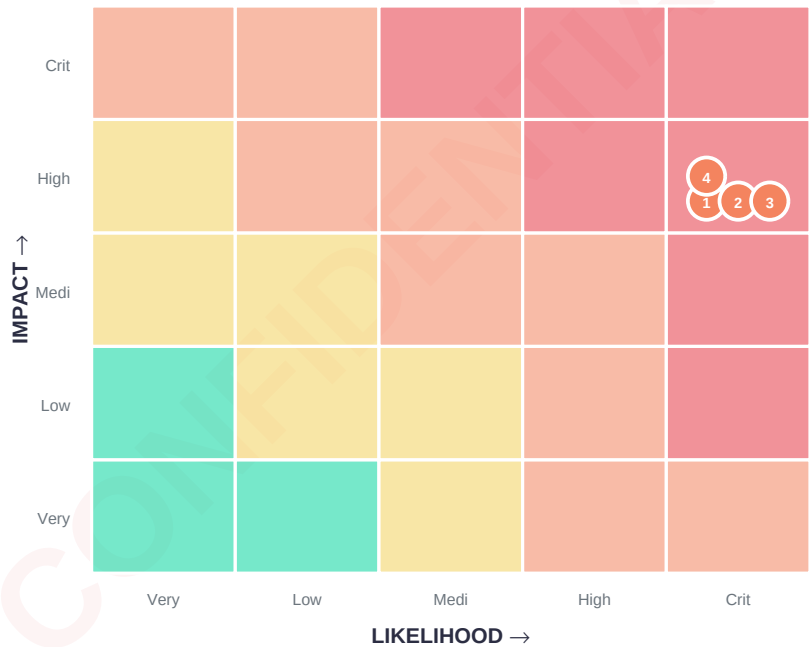
OWASP Top 10 Coverage Heatmap

Intensity reflects how many findings map to each OWASP 2021 category.

A01 Broken Access Control 1	A02 Cryptographic Failures 0	A03 Injection 1	A04 Insecure Design 0	A05 Security Misconfiguration 0
A06 Vulnerable Components 0	A07 Auth Failures 0	A08 Integrity Failures 0	A09 Logging Failures 0	A10 SSRF 0

Risk Matrix — Likelihood × Impact

Each numbered dot is a finding (by its reference), positioned by severity. Upper-right (red) is the highest priority.



Vulnerability Summary



Total findings: 4

Penetration Testing Report

Date: 2026-06-20
Prepared by: RMW-PenStrike
Classification: Client Confidential

Document Control

Client Confidentiality
This document contains Client Confidential information and may not be copied without written permission.

Document Version Control

Issue No.	Issue Date	Issued By	Change Description
0.1	2026-06-20	RMW-PenStrike	Draft for internal review
1.0	2026-06-20	RMW-PenStrike	Released to client

Executive Summary

RMW-PenStrike was engaged to perform a web application penetration test against the target application hosted at <https://google-gruyere.appspot.com>. The objective of this assessment was to identify and exploit security vulnerabilities that could compromise the confidentiality, integrity, or availability of the application and its underlying data. During the engagement, multiple high-severity issues were discovered, including hard-coded

credentials exposed in the login page, a path traversal allowing access to internal secrets, a reflected cross-site scripting vulnerability due to unsafe use of `eval()`, and an insecure authorization mechanism that permits unprivileged users to gain author-level access. These findings collectively pose a significant risk, as they could be chained to achieve full compromise of user sessions, data theft, and privilege escalation. Immediate remediation of the identified vulnerabilities is strongly recommended.

Assessment Summary

The overall risk level for the application is **HIGH**.

Multiple high-severity vulnerabilities were identified during testing. An attacker could leverage these weaknesses to steal sensitive information, take control of user sessions, and escalate privileges, ultimately undermining the security posture of the application and its underlying infrastructure. Prompt action is required to remediate these issues to prevent exploitation.

Phase	Description	Critical	High	Medium	Low	Total
1	Web/API Penetration Testing	0	4	0	0	4
	Total	0	4	0	0	4

Strategic Recommendations

- **[HIGH]** Remove hardcoded credential from the login page source; store secrets in a secure secrets manager or environment variable.
- **[HIGH]** Apply strict path whitelisting and input sanitization to prevent path traversal attacks.
- **[HIGH]** Eliminate the use of `eval()` and any other dynamic code evaluation mechanisms; implement safe output encoding to prevent cross-site scripting.
- **[HIGH]** Move all authorization logic to the server side; do not trust client-supplied parameters such as `is_author` for access control decisions.

1. Technical Summary

1.1 Scope

The following target was in scope for this penetration test:

- <https://google-gruyere.appspot.com>

The assessment focused on the live Gruyere codelab application instance accessible via dynamic subpaths (e.g., <https://google-gruyere.appspot.com/416902709325584161180924204263102047586/>) and its accompanying static documentation.

1.2 Post Assessment Clean-up

Any test accounts or artefacts created during the assessment (e.g., testuser123, attacker) should be disabled or removed to prevent unauthorised access after the engagement. All uploaded files and modified data must be purged.

1.3 Risk Ratings

#	Risk Rating	CVSSv3 Score	Description
1	CRITICAL	9.0 - 10	Requires resolution as quickly as possible.
2	HIGH	7.0 - 8.9	Requires resolution in a short term.
3	MEDIUM	4.0 - 6.9	Should be resolved throughout ongoing maintenance.
4	LOW	1.0 - 3.9	Should be addressed as part of routine maintenance.
5	INFO	0 - 0.9	Reported for information only.

1.4 Findings Overview

Ref	Description	Risk
PT-1-1	Hardcoded Credentials in Login Page	HIGH
PT-1-2	Path Traversal to Sensitive File	HIGH
PT-1-3	Reflected Cross-Site Scripting via eval() Injection	HIGH
PT-1-4	Privilege Escalation via Client-Side Authorization	HIGH

2. Technical Details

2.1 Hardcoded Credentials in Login Page — HIGH

Ref ID: PT-1-1

The login page served at `https://google-gruyere.appspot.com/416902709325584161180924204263102047586/login` contains a hardcoded password string directly embedded in the HTML source and visible on the rendered page. The string `</td><td>` appears between the “Password:” label and the password input field. Because this

credential is exposed to any user who views the page, an attacker can reuse it to authenticate without any prior compromise. This violates the principle of secure secret storage and can lead to unauthorised access to sensitive functionality.

Vulnerability Details

Field	Value
Affects	https://google-gruyere.appspot.com/416902709325584161180924204263102047586/login
Parameter(s)	N/A (embedded in HTML body)
Attack Vectors	Any unauthenticated visitor can view the page source to obtain the hardcoded password.
References	https://cwe.mitre.org/data/definitions/798.html

Evidence

The following snippet was observed in the page source:

```
<span class="label">Password:</span></td><td><input type='password' name='pw'>
```

The literal text `</td><td>` is displayed before the password field and is visible without any inspection.

Remediation Guidance

1. Remove all hardcoded credentials from source code, templates, and configuration files.
2. Store secrets in a dedicated vault (e.g., HashiCorp Vault, AWS Secrets Manager) or at minimum in environment variables referenced by the application.
3. Rotate the exposed credential immediately and audit the application for any other instances of hardcoded secrets.
4. Implement a secure password reset mechanism if the exposed credential is still in use.

2.2 Path Traversal to Sensitive File — HIGH

Ref ID: PT-1-2

The application does not properly sanitise user-controlled path components, allowing an attacker to traverse the filesystem. By sending a request to `https://google-gruyere.appspot.com/123/%2e%2e%2fsecret.txt` (where 123 is an arbitrary snippet identifier and `%2e%2e%2f` decodes to `../`), the server returned the contents of `secret.txt`. This file contained the instance identifier `469885670341739636912374344282967559918` and a base64-encoded URL `/469885670341739636912374344282967559918`. Unauthorised file disclosure can expose sensitive configuration data, credentials, or internal application logic, and may be a stepping stone to further attacks.

Vulnerability Details

Field	Value
Affects	https://google-gruyere.appspot.com/123/%2e%2e%2fsecret.txt
Parameter(s)	Snippet ID path segment (user-controlled)
Attack Vectors	URL-encoded traversal sequences such as <code>../</code> can be used to read arbitrary files below the web root.
References	https://cwe.mitre.org/data/definitions/22.html

Evidence

The following command was used to retrieve the file:

```
curl -s --path-as-is -L -o secret.txt -w "%{http_code}"  
"https://google-gruyere.appspot.com/123/%2e%2e%2fsecret.txt"
```

HTTP response status: 200

File contents: 469885670341739636912374344282967559918 and
/469885670341739636912374344282967559918

Remediation Guidance

1. Implement a whitelist of allowed characters for path parameters and reject any input containing traversal sequences (e.g., `../`, `..\`, URL-encoded variants).
2. Use platform-specific APIs that resolve file paths safely (e.g., `realpath()` in PHP) and validate that the resolved path stays within the intended directory.
3. Consider isolating uploaded content and application data from system files, and serve static content without directly referencing filesystem paths from user input.
4. Configure the web server to deny access to sensitive file extensions and directories by default.

2.3 Reflected Cross-Site Scripting via eval() Injection — HIGH

Ref ID: PT-1-3

The JavaScript file at

<https://google-gruyere.appspot.com//416902709325584161180924204263102047586/lib.js> contains a function `_refresh()` that retrieves the response from the `/feed.gtl` endpoint and passes it directly to `eval()`. The endpoint accepts a `uid` parameter without proper encoding, and the response is a JSON array wrapped in a function call `_feed(...)`. An attacker can craft a `uid` value that breaks out of the JSON context and injects arbitrary JavaScript, which will then be executed in the context of any user viewing the feed. This reflected XSS vulnerability can be used to hijack user sessions, steal credentials, or deface the application.

Vulnerability Details

Field	Value
Affects	https://google-gruyere.appspot.com/416902709325584161180924204263102047586/feed.gtl?uid=325584161180924204263102047586/feed.gtl?uid=
Parameter(s)	uid
Attack Vectors	Injecting "; alert(1);// or similar payloads that terminate the JSON string and insert executable code.
References	https://cwe.mitre.org/data/definitions/79.html

Evidence

The vulnerable code path in `/416902709325584161180924204263102047586/lib.js`:

```
_refresh("/" + uniqueId + "/feed.gtl?uid=" + uid, ...)
```

The `_refresh` callback passes the server response directly to `eval()`. A request such as:

```
https://google-gruyere.appspot.com/416902709325584161180924204263102047586/feed.gtl?uid=test%22;alert(1);//
```

results in the response containing the injected payload, which is then evaluated.

Remediation Guidance

1. Avoid using `eval()`, `Function()`, or any similar dynamic code execution on client-controlled data.
2. If the feed must return structured data, use JSON parsing with `JSON.parse()` instead of relying on `eval()`.
3. Encode all user-supplied output according to the context (HTML entity encoding, JavaScript string escaping) before inserting it into the DOM.
4. Implement a Content Security Policy (CSP) that disallows inline scripts and `eval()` to provide defence in depth.
5. Validate and sanitise the `uid` parameter on the server side to reject unexpected characters.

2.4 Privilege Escalation via Client-Side Authorization — HIGH

Ref ID: PT-1-4

The account registration page at <https://google-gruyere.appspot.com/416902709325584161180924204263102047586/newaccount.gtl> contains a hidden form field, `is_author`, with a value of `True`. Because the application relies solely on this client-controlled parameter to assign author privileges, an attacker can create a new account with full author-level access simply by including `is_author=True` in the sign-up request. This bypasses any intended administrative approval process and grants unauthorised access to features such as file uploads, potentially leading to further exploitation.

Vulnerability Details

Field	Value
Affects	https://google-gruyere.appspot.com/416902709325584161180924204263102047586/newaccount.gt1
Parameter(s)	is_author (hidden form field)
Attack Vectors	Modifying the sign■up request to set is_author=True
References	https://cwe.mitre.org/data/definitions/863.html

Evidence

The sign■up form HTML contains:

```
<input type='hidden' name='is_author' value='True'>
```

A direct request to the action URL with is_author=True and valid user credentials successfully created an account with author privileges, as confirmed by the presence of the “Upload” link in the navigation menu after login.

Remediation Guidance

1. Never rely on client■side fields for authorization decisions; all access control must be enforced on the server side.
2. Remove the is_author parameter from the registration form entirely and assign roles only through a secure administrative interface.
3. Implement proper role■based access control (RBAC) that verifies the authenticated user's permissions before granting access to privileged functionality.
4. Audit the entire application for similar hidden fields or parameters that could be manipulated to elevate privileges.

3. Appendices

3.1 Penetration Testing Methodology

This assessment followed the OWASP Testing Guide v4 and covered the OWASP Top 10 (2021) vulnerabilities. The engagement was structured in four phases:

1. **Reconnaissance** – Passive and active information gathering to map the target.
2. **Enumeration** – Identification of endpoints, technologies, and potential attack surfaces.
3. **Exploitation** – Manual exploitation of identified vulnerabilities with appropriate tooling.
4. **Reporting** – Documentation of findings, evidence, and remediation guidance.

The OWASP Top 10 (2021) table below summarises the primary risk categories considered during testing:

Category	Description
A1 – Broken Access Control	Failures that allow users to act outside of their intended permissions.
A2 – Cryptographic Failures	Weaknesses in protecting data in transit or at rest.
A3 – Injection	Flaws that allow untrusted data to be interpreted as commands or queries.
A4 – Insecure Design	Missing or ineffective security controls due to design flaws.
A5 – Security Misconfiguration	Improperly configured security settings or defaults.
A6 – Vulnerable and Outdated Components	Using components with known vulnerabilities.
A7 – Identification and Authentication Failures	Issues with identity management, authentication, and session handling.
A8 – Software and Data Integrity Failures	Failures to verify the integrity of software updates, CI/CD pipelines, or sensitive data.
A9 – Security Logging and Monitoring Failures	Insufficient logging, detection, and response capabilities.
A10 – Server-Side Request Forgery (SSRF)	Flaws allowing an attacker to make requests on behalf of the server.

CONFIDENTIAL

Threat Intelligence & Public Exploits

Network was unavailable during generation: live CVE/PoC lookup was skipped. Pre-built search links are provided so you can verify manually.

PT-1-1 — Hardcoded Credentials in Login Page

CWE: CWE-798 | **Severity:** HIGH

Search PoC / exploits:

[ExploitDB](#) • [GitHub](#) • [Metasploit / Rapid7](#)

PT-1-2 — Path Traversal to Sensitive File

CWE: CWE-22 | **Severity:** HIGH

Search PoC / exploits:

[ExploitDB](#) • [GitHub](#) • [Metasploit / Rapid7](#)

PT-1-3 — Reflected Cross-Site Scripting via eval() Injection

CWE: CWE-79 | **Severity:** HIGH

Search PoC / exploits:

[ExploitDB](#) • [GitHub](#) • [Metasploit / Rapid7](#)

PT-1-4 — Privilege Escalation via Client-Side Authorization

CWE: CWE-863 | **Severity:** HIGH

Search PoC / exploits:

[ExploitDB](#) • [GitHub](#) • [Metasploit / Rapid7](#)

Remediation Roadmap

Prioritised action plan for development and operations teams. Priority and target timeframe are derived from each finding's severity.

Ref	Vulnerability	Severity	Priority	Target	Status
PT-1-1	Hardcoded Credentials in Login Page	HIGH	P2 · Urgent	< 1 week	■ Open
PT-1-2	Path Traversal to Sensitive File	HIGH	P2 · Urgent	< 1 week	■ Open
PT-1-3	Reflected Cross-Site Scripting via eval() Injection	HIGH	P2 · Urgent	< 1 week	■ Open
PT-1-4	Privilege Escalation via Client-Side Authorization	HIGH	P2 · Urgent	< 1 week	■ Open

CONFIDENTIAL

Document Integrity

The SHA-256 fingerprint below is computed over the report's source content. Any modification of the findings changes this value, allowing the recipient to verify the report has not been altered after delivery.

Report title	Penetration Testing Report
Generated	20/06/2026
Findings	4
Classification	CONFIDENTIAL
SHA-256	bf66e6d0b86a3464bb5c769c575717ecba068a3314a5f746e959a64ed580f48d

Verify with: sha256sum <source>

CONFIDENTIAL